

Manual de ejecución

Tarea 1 - Integración y análisis de recorridos con Apache Spark

Autor: Adrian Rojas Rivera **Modalidad:** individual.

El programa integra tres CSV, calcula agregaciones intermedias y presenta dos rankings por provincia: kilómetros totales y promedio diario. Incluye 33 pruebas con pytest.

1. Preparar el contenedor

Requisito: Docker instalado y en ejecución. Descomprime la entrega y abre una terminal en la carpeta **T1**, donde está el archivo **Dockerfile**. La primera construcción requiere acceso a Internet para obtener la imagen base y sus dependencias.

```
docker build -t tarea1-bigdata .  
docker run --rm -it tarea1-bigdata /bin/bash
```

El segundo comando abre una consola dentro del contenedor. Los siguientes comandos se ejecutan en esa consola:

```
source /opt/venv/bin/activate  
cd /src  
spark-submit main.py data/ciclista.csv data/ruta.csv data/actividad.csv
```

La salida se muestra en la terminal: entradas, datos unidos, totales por persona/provincia/fecha, totales diarios por persona, promedios y los dos rankings. Los resultados no requieren Jupyter ni un servicio web.

2. Ejecutar las pruebas

```
pytest
```

El resultado esperado es **33 passed**. Se debe ejecutar desde **/src** dentro de esta imagen. La sesión de Spark para las pruebas se prepara mediante la fixture de **confest.py**.

En el contenedor usado durante la tutoría, la carpeta compartida aparece como **/t1**. Ese montaje es una comodidad de desarrollo; una copia nueva de la entrega utiliza **/src**, tal como indica el Dockerfile.

Entorno comprobado: Linux ARM64, Java 17, Python 3.10, Spark 4.2.0 y pytest 9.1.1. El Dockerfile procede del material del curso y usa la imagen base eclipse-temurin:17-jdk-jammy.

Entradas y reglas

3. Formato de los tres archivos

CSV en UTF-8, separados por comas, **sin encabezado** y con punto decimal. El orden de los argumentos y de las columnas es obligatorio:

Archivo	Columnas, en este orden
ciclista.csv	Cédula numérica; nombre completo; provincia.
ruta.csv	Código numérico; nombre de ruta; kilómetros decimales.
actividad.csv	Código de ruta; cédula; fecha YYYY-MM-DD.

Se incluyen **17 ciclistas**, **8 rutas** y **21 actividades ficticias**. Los identificadores son únicos en los catálogos de ciclistas y rutas. Se esperan fechas válidas con formato ISO. Las fechas se conservan como texto y se agrupan completas.

Ejemplo de ruta: 100,Ruta Centro,10.5

Ejemplo de actividad: 100,1,2026-10-01

4. Criterios adoptados y documentados

Unión. Dos uniones left conservan a los ciclistas sin actividades. Los campos sin correspondencia quedan en NULL. La unión de los datos incluidos contiene 22 filas: 21 recorridos y la fila de Marta sin actividades.

Recorridos repetidos. Dos filas iguales de actividades cuentan como dos recorridos. No se eliminan ni se sustituyen por un único registro.

Promedio diario. Primero se suma por persona y fecha; después se promedian sus totales diarios. Se consideran los días con actividades de distancia conocida. No se agregan días de calendario sin recorridos, porque el enunciado no define un período.

Personas sin actividad y nulos. Permanecen en la unión, pero se excluyen de los cálculos de distancia y de los rankings. No se inventan kilómetros ni promedios. Una distancia de cero sí es un dato conocido. Cada agrupación omite las claves faltantes que necesita.

Empates. El ranking usa dense_rank por provincia, como el ejemplo del profesor. Se incluyen los N primeros rangos, no un máximo estricto de N personas. Un empate en el corte puede producir más filas.

N. El programa utiliza N=5; las funciones admiten cualquier entero positivo. Rechazan cero, negativos y otros tipos mediante ValueError. Los archivos vacíos conservan su esquema y no causan divisiones por cero.

Resultados que verificar

5. Ejemplo del promedio de Ana

Fecha	Recorridos	Total diario
2026-10-01	10.5 + 10.5 km	21.00 km
2026-10-03	20.25 km	20.25 km

Total: 41.25 km. **Promedio diario:** $(21.00 + 20.25) / 2 = 20.625$ km. Promediar las tres actividades daría 13.75 km por recorrido y no respondería la pregunta del promedio diario.

6. Totales del conjunto de ejemplo

Provincia	Kilómetros
Alajuela	300.25
Cartago	41.25
San José	342.50
Total general	684.00

Los totales por fecha son **601.50** km el 1 de octubre, **51.75** km el 2 de octubre y **30.75** km el 3 de octubre. Su suma también es 684.00 km.

7. Dos rankings distintos en San José

Rango	Por total de kilómetros	Por promedio diario
1	Diego Solís - 80.00	Diego Solís - 80.00
2	Elena Castro - 70.50	Fabio Rojas - 50.00
3	Fabio Rojas - 50.00	Gabriela León - 40.00
4	Ana Mora - 41.25	Elena Castro - 35.25
5	Gabriela León - 40.00	Hugo Soto - 30.00

En Alajuela, María Salas y Néstor Ruiz comparten el quinto rango con 30 km: se incluyen ambos y aparecen seis personas. En Cartago, Rafael lidera por total y Paula por promedio. Cada tabla final contiene 13 personas con este ejemplo.

Pruebas y organización

8. Comprobaciones por responsabilidad

Archivo de pruebas	Cantidad	Qué comprueba
test_tarea1.py	3	Lecturas, columnas, valores y repeticiones.
test_uniones.py	5	Correspondencias, left/inner, repeticiones y ausencia de actividad.
test_casos_vacios.py	5	CSV vacíos y conjuntos sin correspondencia.
test_agregaciones.py	9	Totales, homónimos, días completos, promedios, nulos y cero.
test_rankings.py	7	Top por provincia, N variable, empates, vacíos y métricas nulas.
test_programa.py	4	Argumentos y resultados del ejemplo completo.

Las pruebas de agregación parten de DataFrames intermedios en memoria; las de ranking parten de métricas preparadas. Así se cumple la independencia solicitada en la rúbrica. La comprobación completa complementa esas pruebas, sin sustituirlas.

9. Archivos de la entrega

main.py coordina la ejecución. **functions/functions.py** contiene las funciones reutilizables. **confest.py** prepara Spark. **data/** contiene el ejemplo completo y **tests/data/** los archivos pequeños de prueba. **Dockerfile** construye el entorno. **README.md** resume la ejecución.

Las funciones principales son `load_cyclists`, `load_routes`, `load_activities`, `join_data`, `calculate_total_kilometers`, `calculate_province_totals`, `calculate_day_totals`, `calculate_daily_totals`, `calculate_daily_average` y las dos funciones `get_top_cyclists`. El promedio recibe totales diarios; los rankings reciben agregados por ciclista.

10. Fuentes y observaciones

Se adaptaron el enunciado 2026-02-M4-T1, el FAQ, `basic/read.py`, `Spark_DataFrames_API.ipynb` y el ZIP de solución compartido por el profesor. El Dockerfile es una copia del Dockerfile.arm64 del curso. La atribución y las adaptaciones se detallan en **doc/Fuentes.md**.

El desarrollo contó con apoyo de Codex para explicación, adaptación, documentación y pruebas. La política específica de IA citada por el curso no fue aportada; el autor debe revisar cualquier requisito adicional antes de entregar. La guía de estudio permite repasar y explicar la implementación.

Si aparece un error de argumentos, comprueba que se suministren tres rutas CSV existentes. Si Docker no conecta, abre Docker Desktop. Ejecuta desde el directorio del proyecto dentro del contenedor. Los mensajes de cierre de Spark después de imprimir resultados corresponden al cierre de su sesión.